Testing, tooling & Colosseum readiness.

FROM A CLEAN CLONE TO A DEMO THAT JUDGES REMEMBER.

SESSION GOALS

Turn your hackathon codebase into something judges can run, verify, and trust — without help from you.

The usability criterion, stated plainly

> Usability: How quickly a developer can go from "zero" to a "functional dApp" using the toolkit.

Translation: a cold judge, on a clean machine, should clone your repo and reach a live transfer in **under 10 minutes**.

Start from the official scaffold

```
npx create-solana-dapp@latest
```

- Official, maintained by Solana Developers.
- Produces a working Next.js or React app.
- Includes wallet adapter and a starter program.

> Your toolkit should either **wrap** this or **replace** it with something equally low-friction. Never reinvent.

Source: create-solana-dapp

The three environments you will use

LOCALNET `solana-test-validator` for deterministic testing.

DEVNET For realistic integration, with a free faucet.

MAINNET-BETA For the final judged demo when appropriate.

- Your **.env.local** controls cluster + RPC.
- Switching clusters should require **zero code changes**.

If a single `cp .env.example .env.local` is the difference between "runs" and "crashes", your toolkit is not plug-and-play.

The frontend test plan

TEST	WHY
Connect & disconnect a wallet	Provider wiring is correct.
Reject a signature	Your UI handles the soft-no path.
Submit with a bad primary RPC	Fallback kicks in.
Let a transaction expire	Expired state triggers, retry works.
Run on a phone	Mobile readiness and honesty checks.
Verify on Explorer	Signatures match on-chain reality.

Don't ship without these

- **A live Devnet URL** (Vercel, Netlify, or equivalent).
- **A README** that lists install + run commands.
- **Env variables** documented with examples.
- **A demo wallet with Devnet SOL** for judges, or clear faucet instructions.
- **Explorer links** in every success state.

> A locally-running demo that a judge cannot clone and run is **not** a submission.

Mobile, realistically

WHAT WORKS

- Desktop browser with Phantom / Solflare.
- Android with Mobile Wallet Adapter.

WHAT DOESN'T

- iOS MWA (no official support yet).
- In-wallet webview flows that promise more than they deliver.

> Colosseum-ready = desktop is flawless, Android is tested, iOS shows a factual notice.

Source: docs.solanamobile.com

A ten-minute judging ritual

0–3 MIN

Clone, install, copy `.env`, run `npm run dev`. Expect it to just work.

3–7 MIN

Connect a wallet, fund it from the Devnet faucet, send a transfer.

7–10 MIN

Verify on Solana Explorer. Poke the edge cases.

> If your project survives this ritual, you are in the top quartile of hackathon submissions.

The toolkit we ship with this module

- **Next.js 14 + Wallet Adapter + web3.js 1.x** starter.
- Real **SOL transfer** page with the 5-stage lifecycle.
- Real **account inspector** page reading live data.
- **RPC fallback** built in.
- Priority fee helper.
- Editorial UI, WCAG-conscious, Ukrainian-authored.

> Location in the submission: `toolkit/starter/`. 1375 packages install, typechecks clean, builds clean.

HANDS ON

You will ship a Colosseum-ready app using `create-solana-dapp` and this module's toolkit.

Assignment 4 — live recap

- Scaffold with `create-solana-dapp`.
- Wire a real on-chain action (transfer or counter).
- Implement confirmation-safe flow with explicit states.
- Disclose mobile reality honestly.
- Ship a public Devnet URL + a README judges can follow.

> Full spec: `assignments/assignment-4.md`.

Quiz 4 — topics covered

- The official scaffold command and when to use it.
- Devnet vs localnet vs mainnet-beta.
- What production requires beyond a happy-path demo.
- Mobile Wallet Adapter support matrix.
- The ten-minute judging ritual.

> Full quiz with answers: `quizzes/quiz-4.md`.

END OF THE MODULE

You are now equipped to ship a frontend judges remember.

References

- create-solana-dapp — github.com/solana-developers/create-solana-dapp
- Frontend docs — solana.com/docs/frontend
- Production readiness — solana.com/docs/payments/production-readiness
- Confirmation — solana.com/developers/guides/advanced/confirmation
- Solana Mobile — docs.solanamobile.com